

第2回レポート「一筆書きプログラム」の説明資料

1. グラフの一筆書き問題を解く Prolog プログラムの作成

グラフの辺の一筆書きは, グラフの 1 つの頂点からスタートしてすべての辺を 1 回ずつたどるような経路を計算する問題です. この問題を解く Prolog プログラムを作ります. 方針としては, 前回の宿題で作成した「迷路を解くプログラム」を利用します.

迷路を解くプログラムでは, 次のように next ルールを定義すると, 実行が停止しなくなるという問題がありました.

```
next(X, P, Y) :- neighbor(X, P, Y).  
next(Y, P, X) :- neighbor(X, P, Y).
```

この問題は, 前回の授業で説明された, fd_... で始まるルールを使うことで解決できます. これらのルールは変数にバインドする値を制限する働きがあります. 例えば, 「値の範囲を決める」ルールを使用すると, その範囲内の値だけを変数にバインドして単一化を進めます. また, 「リストの全ての値が異なる」ルールを利用すると, すでに求めたリストの中の値とは別の値を変数にバインドします. したがって, 今回作成するプログラムの作成方針として, 前回作った迷路を解くプログラムとほぼ同じように事実とルールを書きつつ, fd_... のルールを利用するように修正を加えます.

この資料では, 次のグラフ 1 の一筆書きを考えます.

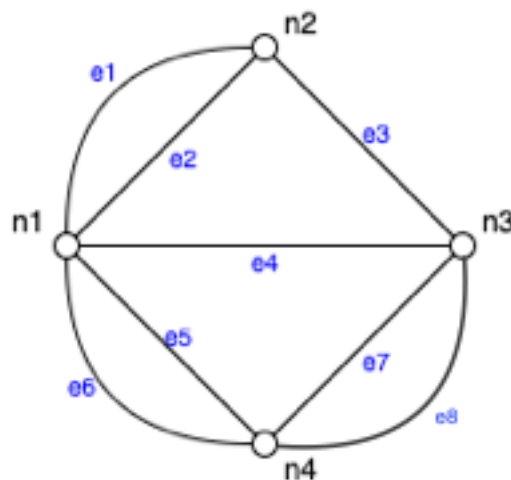


図1 グラフ 1

1.1 プログラムの全体構成

これから `euler1.pl` を作成します. これは前回の迷路を解くプログラムとほとんど同じです. 鍵になるのは, グラフの接続情報を記載する事実の書き方と, `fd_..` の使いどころです.

```
%%% グラフの接続情報
(1.2 節で説明)

%%% 辺を辿るルール next と walk
(1.3 節で説明)

%%% 一筆書きルール eulerTrail
(1.4 節で説明)
```

1.2 事実の記述(グラフの接続情報の記述)

前回のプログラムを参考に, グラフの点と辺の接続情報を事実として記述します. ルール `fd_..` を使うときには変数には, 整数などの定数をバインドする必要があります. そこで, 次の規則にしたがって, 点と辺の名前を整数に変換することにします.

- 点を表す整数: 点の番号に 100 を足した整数
 - 例) 点 `n1` → 101
- 辺を表す整数: 辺の番号そのまま
 - 例) 辺 `e1` → 1

この規則にしたがって, グラフ 1 の接続情報を事実 `neighbor` で記述してみます.

```
neighbor(101, 1, 102).
neighbor(101, 2, 102).
neighbor(102, 3, 103).
neighbor(101, 4, 103).
neighbor(101, 5, 104).
neighbor(101, 6, 104).
neighbor(103, 7, 104).
neighbor(103, 8, 104).
```

1.3 ルール next と walk

ルール `next` と `walk` は前回の宿題から変更の必要はありません. なお, `next` は辺をどちらからでも辿れるように 2 行書きます.

```
...
next(X, P, Y) :- neighbor(X, P, Y).
next(Y, P, X) :- neighbor(X, P, Y).
```

```
walk ... (省略) ....
```

1.4 一筆書きルール eulerTrail

一筆書きで描いた経路のことをオイラー路 (Euler Trail) と言います. そこで, 一筆書きのルールには eulerTrail という名前をつけることにします.

ルール eulerTrail は次のことを意味します.

```
eulerTrail(U, L, V) ... 点 U から経路 L を通って, 点 V に到るオイラー路がある
```

L は一筆書きでたどる辺の整数のリストです. グラフ 1 の場合, 辺は e1~e8 の 8 個で, 一筆書きではすべての辺を 1 回ずつ通ることになりますので, リスト L の長さは 8 で, L 中の整数の範囲は 1~8 です.

ルール eulerTrail の定義は次のように書けます.

```
eulerTrail(U, L, V) :- ① , ② , ③ , ④
```

- ① 「L の長さは辺の個数と等しい」 (ルール length を使う)
- ② 「L の要素の範囲..」
- ③ 「L の要素はすべて互いに異なる」
- ④ walk ルール

なお, ルール length(L, N) は「リスト L の長さは N に等しい」ことを意味するルールです. これは前回紹介したルール count とほぼ同じだが, 今回のプログラムでは, count と fd... を組み合わせると停止しないことがあるため, ①では count ではなく length を使用すること.

1.5 やってみよう

グラフ 1 のオイラー路を求める Prolog プログラム euler1.pl を作ろう. ここまでの 1.1 節から 1.4 節の説明を参考に作れば良い.

プログラムが完成したら, 次のことを確かめよう.

- グラフ 1 では n1 から n2 へのオイラー路が存在します. 図で確認してください.
- 次に Prolog プログラムを読み込んで, 次の質問を入力するとオイラー路のリスト L の回答が表示されることを確認してください.

```
| ?- eulerTrail(101, L, 102).
L = [1,3,7,5,4,8,6,2] ? ;

L = [1,3,7,5,6,8,4,2] ? ;

...
```

- グラフ 1 では n1 で始まり, n3 で終わるオイラー路と, 同じく n4 で終わるオイラー路は存在しません. このことを図 1 で確かめてみましょう.
- 次の質問を入力すると, オイラー路が存在しないことを意味する no が回答されます. したがってこれらのオイラー路が存在しないことが実行結果からわかります.

```
| ?- eulerTrail(101, L, 103).

(1 ms) no
| ?- eulerTrail(101, L, 104).

(3 ms) no
```

- 次に n1 で始まるオイラー路の終点を求めてみます. この質問では経路には興味がないので変数 L の代わりにワイルドカードを使用し, 終点を変数 V で表現します. 実行結果は 102 が複数回表示されます. n1 で始まるオイラー路の終点は n2 だけであることがわかります.

```
| ?- eulerTrail(101, , V).
V = 102 ? ;

V = 102 ? ;

...
```

- 今度は n1 から始まり, 同じ点で終わるオイラー路(これをオイラー閉路といいます)を求めてみます. さらに, 任意の点 V から始まるオイラー閉路を求めてみます. どちらも結果は no です, オイラー閉路が存在しないことがわかります.

```
| ?- eulerTrail(101, , 101).

(7 ms) no

| ?- eulerTrail(V, , V).

(7 ms) no
```

(次ページに続く)

2. グラフの全ての点を1回ずつ辿る経路を求める.

グラフの辺を1回ずつ辿る経路, オイラー路については1節で説明しました. 辺ではなく点を1回ずつ辿る経路をハミルトン路と言います.

2.1 ハミルトン路の例

グラフ1のハミルトン路の例を示します.

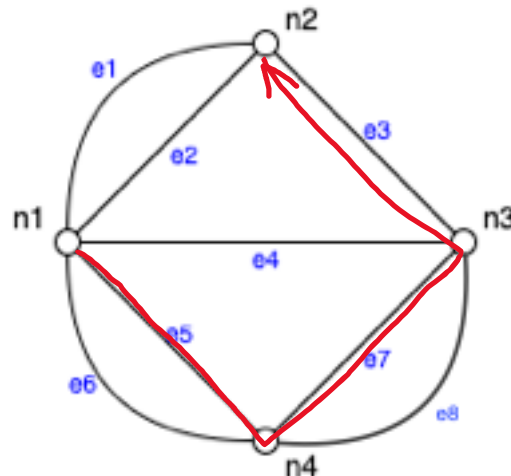


図2: グラフ1のハミルトン路

グラフ1の点n1からn2へのハミルトン路の例:

ハミルトン路: e5, e7, e3

訪れる点: n1, n4, n3, n2

2.2 プログラム hamilton1.pl とグラフの接続情報とルール next

プログラム hamilton1.pl の構成は euler1.pl と同じです.

事実 neighbor とルール next はグラフの接続情報と辺を一つ辿ることを意味します. 今回使うグラフは1節と同じグラフ1とグラフ2で変更はありません. したがって, euler1.pl の neighbor と next をそのまま使います.

2.3 ルール traverse (walk の代わり)

オイラー路のプログラムの walk と区別するため, ハミルトン路のプログラムのルールを traverse と書くことにします. このルールはかなりの変更が必要です. 鍵になるのは, 経路のリストと訪れた点のリストです.

- ハミルトン路を計算する必要があるので, traverse の引数には, 辺のリストを指定する必要がある
- 過去に訪れた点を2度通らないようにするため, 過去に訪れた点のリストを指定する必要がある.

ルール traverse は次の4つの引数を取るものとして定義しましょう.

```
traverse(U, L, M, V) ... 点 U から経路 L, 点列 M を通って点 V に至る経路がある
```

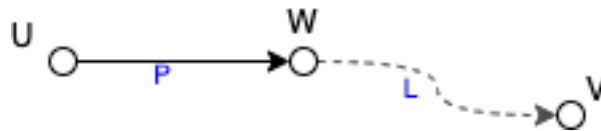
なお, L は点 U から V を辿る間に通る辺のリストです. また, M は点 U から V を辿る間に通る点のリストです. ただし, 点 V は M に含まれ, 点 U は M に含まれないものとします.

ルール traverse の定義は次のように書けます.

```
traverse(U, [], [], U).  
traverse(U, [P|L], [W|M], V) :- ① , ②
```

✧ 1 行目は「点 U からはどこも辿らずに点 U に至る」ことを意味します. 経路 L と点列 M は空リストとします.

✧ 2 行目は次のような経路を辿ることを意味しています.



第 3 引数の M は点 W の後で経路 L を通って V に至る間に通る点のリストです.

① 「点 U から辺 P を通って点 W に行く」

② 「点 W から経路 L, 点列 M を通って点 V に至る経路がある」

2.4 ハミルトン路を求めるルール hamiltonTrail

ルール hamiltonTrail は次のことを意味します.

```
hamiltonTrail(U, L, M, V)  
... 点 U から経路 L と点列 M を通って, 点 V に至るハミルトン路がある
```

ルール traverse との違いは, traverse は点列 M に特に条件がないのに対して, hamiltonTrail は点列 M がグラフの全ての点が 1 つずつ含まれているという点です. 特に traverse の点列 M は始点 U を含みませんが, hamiltonTrail では M に点 U が含まれるように定義します.

ルール hamiltonTrail の定義は次のように書けます.

```
hamiltonTrail(U, L, [U|M], V) :- ① , ② , ③ , ④
```

① 「リスト [U|M] の長さは点の個数と等しい」

- ② 「リスト[U|M]の要素の範囲」
- ③ 「リスト[U|M]の要素は互いに異なる」
- ④ traverse ルールを使う

2.5 やってみよう

グラフ 1 のハミルトン路を計算するプログラム `hamilton1.pl` を作ってみよう. 完成したら次のことを調べてみよう.

- Prolog で図 2 の赤色のハミルトン経路を計算する質問を入力しよう.

```
| ?- hamiltonTrail(101, L, M, 102).  
  
L = [5,7,3]  
M = [101,104,103,102] ? ;  
  
L = [5,8,3]  
M = [101,104,103,102] ? ;  
  
no
```

- 2 つの経路が回答されました. 図 2 と比べてみよう
- 次に, グラフ 1 の点 `n1` から始まるハミルトン路の最後の点を求めてみます. この質問では, 辺列と点列には興味がないので変数 `L`, `M` の代わりにワイルドカードを使用します. また, 終点には変数 `V` を指定します.
- 実行結果からは, `n2` のほかに `n1` から `n4` へのハミルトン路があるとわかります.

```
| ?- hamiltonTrail(101, , , V).  
  
V = 104 ? ;  
  
V = 104 ? ;  
  
V = 102 ? ;  
  
V = 102 ? ;  
  
(1 ms) no
```

ここまで終わったら, レポートのテンプレートを開いて, 作ったプログラムや実行結果を記録してレポートを提出しましょう.

(次ページに続く)

3. レポート課題

次のグラフ 2 におけるオイラー路とハミルトン路を求めるプログラムを作成し、質問によってオイラー路、ハミルトン路が存在するかどうかを確認しなさい。

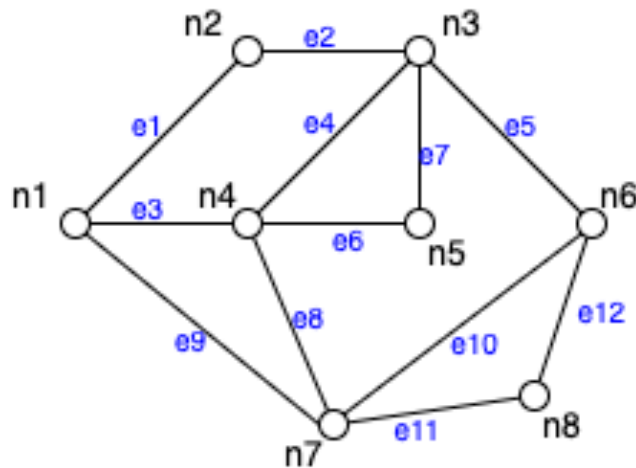


図 3 グラフ 2

レポートは次の構成とすること。

1. 表紙: 講義名とレポート課題名, 学生番号と氏名を記載すること
2. 今回のレポート課題の概要説明 (100 文字程度, 1 点満点)
3. グラフ 2 のオイラー路を求めるプログラム euler2.pl を作成せよ. プログラムは Blackboard で提出し, このレポートにはプログラムの説明文を記述せよ. (3 点満点)
4. グラフ 2 のオイラー路に関する次の四つの質問を Prolog に入力せよ. また, 質問に対する Prolog の回答と, グラフとを見比べて Prolog の回答が正しいことを確認せよ. ここでは四つ質問それぞれに対して, ①入力した質問文, ②Prolog の回答, ③回答の意味の説明, の三つを記述せよ. (3 点満点)
 - ✧ グラフ 2 の点 n1 から n6 へのオイラー路があるか?
 - ✧ グラフ 2 の点 n1 から始まるオイラー路の終点は? (終点を表す変数を使用して 1 つの質問にまとめること. 経路には興味がないのでワイルドカードを使うこと)
 - ✧ グラフ 2 の点 n1 からはじまるオイラー閉路が存在するか? (yes または no を回答するような質問を記述すること)
 - ✧ グラフ 2 にオイラー閉路は存在するか (点を変数 V とおいて, V から V へのオイラー路があるか質問するとよい)
5. グラフ 2 のハミルトン路を求めるプログラム hamilton2.pl を作成せよ. ここでは, プログラムの説明文を記述せよ. (3 点満点)

6. グラフ 2 のハミルトン路に関する次の質問文を Prolog で作成せよ。また、質問に対する Prolog の回答とグラフとを見比べて、Prolog の回答が正しいことを確認せよ。ここでは三つ質問それぞれに対して、①入力した質問文、②Prolog の回答、③回答の意味の説明、の三つを記述せよ。(3 点満点)
- ◇ グラフ 2 の点 n1 から点 n5 へのハミルトン路が存在するか？存在するならば、ハミルトン路の点列を回答させること(辺列には興味がないのでワイルドカードを使用すること)
 - ◇ グラフ 2 の点 n1 から点 n6 へのハミルトン路があるか？存在するならば、ハミルトン路の点列を回答させること(辺列には興味がないのでワイルドカードを使用すること)
 - ◇ グラフ 2 において、n1 を始点とするハミルトン路の終点は？(終点を変数として、ハミルトン路が存在するか質問すると良い)
7. このレポート課題を通して学習したことを説明せよ。(2 点満点)
8. (追加課題)このレポート課題と Prolog の両方に関連するキーワードを 1 つ示して次のことを記述せよ。(最大 3 点)
- キーワードの名称と説明文
 - キーワードの調査方法と URL など参考文献情報
 - キーワードとレポート課題がどのように関連するかの説明
9. 感想(3 点満点)
-

生成 AI の使用ルール

この課題では生成 AI を使用しないでください。また、Web 検索等も不要です。

このレポート課題は授業の内容及びこの資料の内容だけで解答できるように設定されています。このため、生成 AI や Web 検索を使用する必要はありません。むしろこれらを使用することによって、間違った結果になる恐れがありますので、使用しないでください。ただし例外として、追加課題については Web 検索のみ使用を認めます。

もしも課題につまずいたら、まずは授業資料とこの資料を読み返してみましょう。それでも解決方法がわからない場合は先生に相談してください。

以上